

# AutoTraQ Cheat Sheet — Ben (Frontend)

---

## Your Section: Frontend (React + Vite + TypeScript + Tailwind CSS)

---

### Quick Overview

The frontend is a **React single-page application** built with Vite (fast dev server & bundler), TypeScript, and Tailwind CSS for styling. It communicates with the Express backend via a REST API client.

---

### Tech Stack

- **React 18** — component-based UI framework
  - **Vite** — build tool & dev server (replaces Webpack, much faster)
  - **TypeScript** — typed JavaScript (catches bugs at compile time)
  - **Tailwind CSS** — utility-first CSS framework (no separate CSS files)
  - **React Router** — client-side page navigation
  - **Recharts** — charting library (used on Dashboard)
  - **Lucide React** — icon library
- 

### Project Structure

```
frontend/
├── src/
│   ├── main.tsx          ← App entry point
│   ├── App.tsx           ← Routes & top-level layout
│   ├── api/
│   │   └── client.ts     ← API client (all backend calls go through here)
│   ├── components/
│   │   ├── Layout.tsx    ← Main app layout (sidebar, header)
│   │   ├── CommandBar.tsx ← %K search/command palette
│   │   ├── ProtectedRoute.tsx ← Auth guard for pages
│   │   ├── Barcode.tsx   ← Barcode rendering component
│   │   ├── HierarchyBrowser.tsx ← Parts category tree browser
│   │   ├── ImageGallery.tsx ← Image viewer for parts
│   │   ├── NotificationBell.tsx ← Real-time notification icon
│   │   └── PartSearch.tsx ← Search input with autocomplete
```

|                      |                                        |
|----------------------|----------------------------------------|
| SearchStatsCards.tsx | ← Search result stat cards             |
| ConditionBadge.tsx   | ← Part condition color badge           |
| Skeleton.tsx         | ← Loading skeleton placeholders        |
| contexts/            |                                        |
| AuthContext.tsx      | ← Auth state (login/logout/user/token) |
| ThemeContext.tsx     | ← Light/dark theme toggle              |
| pages/               |                                        |
| DashboardPage.tsx    | ← Main dashboard with charts & KPIs    |
| LoginPage.tsx        | ← Login form                           |
| InventoryPage.tsx    | ← Inventory management                 |
| PartsPage.tsx        | ← Parts catalog listing                |
| PartDetailPage.tsx   | ← Single part detail view              |
| PartsSearchPage.tsx  | ← Advanced search with filters         |
| VehiclesPage.tsx     | ← Vehicle management                   |
| RequestsPage.tsx     | ← Part request workflow                |
| CsvPage.tsx          | ← CSV import/export                    |
| ScanPage.tsx         | ← Barcode scanner                      |
| AdminPage.tsx        | ← User management (admin only)         |
| AuditPage.tsx        | ← Audit trail viewer                   |
| SearchPage.tsx       | ← Global search                        |
| vite.config.ts       | ← Vite configuration                   |

## Key Concepts to Know

**1. API Client ( `api/client.ts` )** All backend calls go through one file. Example:

```
const parts = await api.getParts();
const part = await api.getPartById(id);
await api.createPart({ name: 'Brake Pad', sku: 'BP-001' });
```

**2. Auth Flow ( `contexts/AuthContext.tsx` )** - User logs in → gets JWT token → stored in context  
 - `ProtectedRoute` wraps pages that require auth - `useAuth()` hook gives you `user` , `token` , `isManager` , `logout()`

**3. Routing ( `App.tsx` )** - Uses React Router v6 with `<Routes>` and `<Route>` - Protected routes check auth before rendering - Example: `/dashboard` , `/inventory` , `/parts/:id`

**4. Styling (Tailwind CSS)** - No CSS files — styles are class names directly in JSX - Example: `<div className="bg-white rounded-lg shadow p-4 flex items-center gap-2">` - Dark mode: uses `ThemeContext` to toggle classes

## How to Run Locally

```
cd frontend
npm install
npm run dev          # Starts dev server on http://localhost:5173
```

Make sure the backend is running on port 3002 first.

---

## Talking Points for the Meeting

- "The frontend is a **React SPA** with **TypeScript** for type safety and **Tailwind** for rapid UI development"
- "We used **Vite** instead of Webpack for significantly faster build times"
- "The **API client layer** centralizes all backend communication in one file for maintainability"
- "**Auth context** manages JWT tokens and role-based access — admin, manager, fulfillment, and viewer roles"
- "The **Dashboard** uses **Recharts** for data visualization — inventory trends, top movers, dead stock analysis"
- "We implemented **skeleton loading states** for better UX while data loads"
- "The **Command Bar** (⌘K) provides quick search and navigation across the entire app"